

LuxLLM-Agent: An Explainable Opponent-Aware LLM Agent and Demonstration System for Lux AI Season 3

Ze Wang

Project Demonstration Paper

Abstract

This paper presents LuxLLM-Agent, an explainable LLM-assisted agent and demonstration system for Lux AI Season 3. The project is positioned in the broader Lux AI Challenge series, where earlier seasons emphasized resource collection, city or factory management, production chains, spatial expansion, and multi-agent competition. Before finalizing the Season 3 system, this work conducted exploratory comparison tests using Season 1 and Season 2 viewer and agent materials as design baselines. These comparisons showed that previous-season replay or viewer pipelines are useful for visualizing board state, but they are insufficient for explaining the internal reasoning process of an LLM-assisted Season 3 agent.

The final system combines a language-model-assisted intent layer, deterministic fallback execution, cached-intent reuse, persistent game memory, a log-driven GameView, opponent-risk analysis, and local-utility-aware risk filtering. The goal is not to claim state-of-the-art Lux AI performance, but to build a stable and inspectable agent runtime in which observations, memory, LLM decisions, fallback actions, opponent pressure, risk-filter events, and final unit commands can be traced frame by frame. A controlled benchmark of 57 completed matches shows that LLM-assisted agents outperform the rule-only control in the tested setting, while runtime analysis shows that smaller models can be more practical under real-time constraints. Frame-level diagnostics further show that the representative run contains 494 logged viewer frames, 1196 risk-filter events, and 210 changed target events. The local-utility-aware filter reduces far rewrite rate from 0.4160 to 0.0476 and very far rewrite rate from 0.2880 to 0.0190. These results demonstrate a reproducible bridge between LLM-assisted intent generation, opponent-aware perception, action filtering, and post-hoc explanation in a partially observable multi-agent game.

1 Introduction

Game AI environments provide useful testbeds for studying long-horizon decision making, partial observability, multi-agent interaction, and explainable agent behavior. The Lux AI Challenge series is particularly useful for this purpose because each season presents a different strategic abstraction while retaining a common competitive structure: autonomous agents must act under constrained observations, limited resources, and adversarial pressure.

Lux AI Season 3 is challenging because an agent must coordinate multiple units, explore hidden map information, reason about energy and relic-related objectives, and respond to an opponent under limited observation. These properties make Season 3 suitable for studying the interaction between LLM-assisted decision making and classical deterministic control. A language model can provide high-level strategic intent, but the final game action must still be legal, timely, and grounded in the current observation.

Large Language Models can support high-level reasoning and intent generation, but they are not naturally reliable low-level game controllers. Their outputs may be slow, inconsistent, incomplete, or too abstract to map directly to legal actions. LuxLLM-Agent therefore uses the LLM as a high-level decision assistant rather than as a direct action emitter. Deterministic fallback, cached-intent reuse, memory tracking, and action validation keep the runtime stable.

This project focuses on building an explainable agent system. The central question is not only whether the agent can act, but whether the system can show why it acted, what information it

used, and how risk-aware filters changed final target selection. For this reason, the project includes both an agent runtime and a log-driven GameView that reconstructs the agent’s internal decision context.

The work makes four main contributions:

1. It provides an LLM-assisted Lux AI Season 3 agent runtime that combines high-level unit intents, deterministic fallback, cached-intent reuse, and persistent memory.
2. It develops a log-driven GameView that exposes frame-level state, memory, LLM or fallback decision source, unit intents, target lines, warnings, opponent risk, and risk-filter evidence.
3. It compares previous Lux Season 1 and Season 2 viewer or agent materials as design baselines, showing why a Season 3-specific explanation pipeline is necessary.
4. It provides controlled benchmark and diagnostic evidence, including 57 completed matches, frame-level viewer logs, opponent-risk metadata, and local-utility-aware risk-filter improvements.

2 Background: Lux AI Challenge Seasons

The Lux AI Challenge is a series of competitive programming environments for game AI research and engineering. Across different seasons, participants design autonomous agents that operate under constrained observations, limited resources, and adversarial interaction. Although each season has different rules and mechanics, the general challenge remains consistent: an agent must transform partial game observations into coordinated actions over a long-horizon match.

2.1 Season 1: Resource Collection and City Survival

Lux AI Season 1 focused on grid-based resource collection, city expansion, worker coordination, and survival under day-night constraints. Agents controlled units that collected resources and supported city tiles. The strategic problem emphasized classical planning issues such as resource gathering, unit routing, city fuel management, spatial expansion, and survival timing.

From an agent-design perspective, Season 1 is useful because it highlights the importance of spatial planning. Units must move across a grid, collect resources, return them to useful structures, and coordinate with other units. From a viewer-design perspective, Season 1 also demonstrates why visual inspection is valuable: a human developer can quickly understand whether units are exploring efficiently, whether cities are being supplied, and whether spatial congestion is occurring.

However, Season 1-style visualization is usually strongest at showing what happens on the board. It is weaker at explaining why an agent selected a particular action. For a rule-based bot, this may be acceptable because the logic can often be inspected directly in code. For an LLM-assisted agent, however, the viewer must expose additional signals: the decision source, the model or fallback mode, unit-level intent, memory state, and reasons behind target selection.

2.2 Season 2: Factories, Robots, Lichen, and Production Chains

Lux AI Season 2 introduced a different type of strategic structure. Instead of focusing primarily on workers, cities, and survival cycles, Season 2 involved factories, robots, lichen growth, power management, resource collection, and territorial expansion. This setting increased the importance of multi-stage planning. Agents needed to reason not only about movement, but also about production, resource conversion, power constraints, factory placement, and map control.

Season 2 is useful as a comparison point because it shows how the Lux series can move beyond direct unit movement into more complex production and planning systems. A useful viewer for Season 2 needs to make production context visible, not merely unit position. It should help the developer understand why a robot moves, why a factory produces a particular unit, how power constraints influence action choice, and how territorial expansion relates to long-term score.

For the present project, Season 2 comparison tests helped clarify an important point: the relevant information for explanation is not always part of the official board rendering. Strategic context may live inside the agent. This includes plans, priorities, fallback status, resource assumptions, and reasons for selecting particular targets. Therefore, a viewer that only renders the official state cannot fully explain the agent.

2.3 Season 3: Relic Discovery, Energy, Hidden Information, and Opponent Pressure

Lux AI Season 3 changes the setting again. The environment introduces relic discovery, energy-based unit movement, hidden map exploration, multi-unit coordination, and opponent interaction under partial observability. In this setting, an agent must remember discovered areas, infer useful relic-related locations, coordinate multiple units, and react to visible enemy pressure.

These properties make Season 3 especially suitable for explainable LLM-agent research. The game state is not fully observable, so the agent’s memory matters. The task is not a single immediate action, so high-level intent matters. The opponent can create local pressure, so risk analysis matters. The action planner must translate intent into legal moves, so deterministic grounding matters. Finally, because LLM decisions can be opaque, the viewer must show how LLM output, fallback behavior, memory, and risk filtering interact.

This project therefore focuses on Season 3 rather than earlier seasons. The goal is not to reuse a Season 1 or Season 2 viewer directly, but to build a log-driven explanation system for the Season 3 agent loop. The final system records observations, memory summaries, LLM or fallback decision source, opponent-risk information, risk-filter events, and final unit actions. This allows the Season 3 agent to be inspected frame by frame rather than only evaluated through final match outcome.

Table 1: High-level comparison of Lux AI Challenge seasons and their relevance to this project.

Season	Main Characteristics	Relevance to This Project
Season 1	Grid-based resource collection, worker coordination, city expansion, and survival under day-night constraints.	Establishes the importance of spatial planning, movement visualization, and replay inspection.
Season 2	Factory-based production, robot control, lichen growth, power management, resource conversion, and territorial expansion.	Shows that agent explanation must include planning context and resource constraints, not only final unit movement.
Season 3	Relic discovery, energy-based movement, hidden map exploration, multi-unit coordination, and visible opponent pressure.	Provides the final environment for this project because it requires memory, LLM-assisted intent, opponent-aware risk analysis, and risk-aware filtering.

3 Cross-Season Viewer and Agent Comparison

Before finalizing the Season 3 system, this project conducted exploratory comparison tests across earlier Lux AI Challenge settings. The purpose of these tests was not to build competitive agents for all seasons, and it was not to compare win rates across different games. Instead, the tests were used to understand what could and could not be transferred from previous-season viewers and agent pipelines into a Season 3 LLM-assisted agent.

3.1 Purpose of the Comparison

The cross-season comparison focused on three questions:

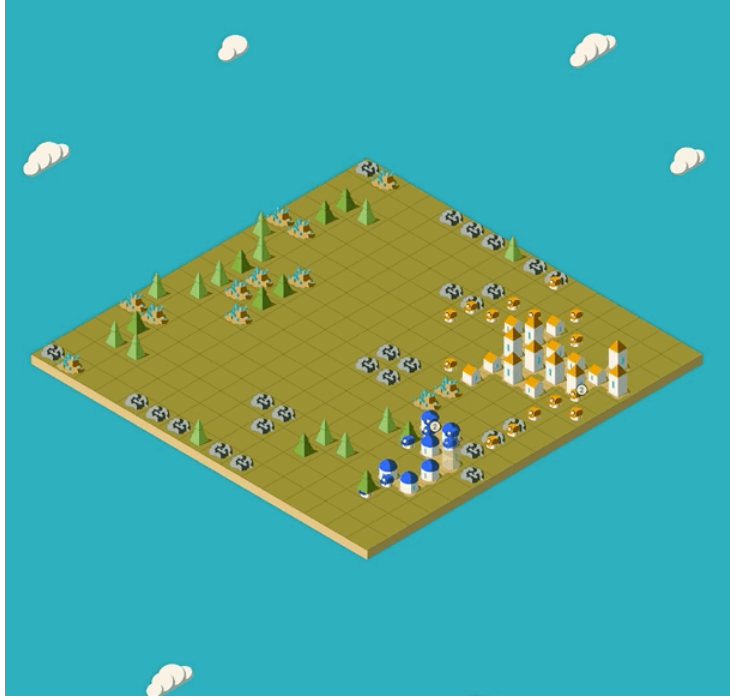


Figure 1: Season 1 viewer comparison screenshot from the exploratory cross-season test. The Season 1-style visualization is useful for board-state inspection, grid movement, resources, and replay understanding. However, it does not expose LLM decision source, fallback status, memory summaries, opponent-risk analysis, or unit-level intent.

1. Can an earlier-season viewer provide a useful basis for explaining a Season 3 agent?
2. Which visual concepts transfer across seasons, and which are season-specific?
3. What additional metadata is required when the agent includes an LLM decision layer?

The comparison confirmed that previous viewers are useful for board-level rendering. They can show maps, unit positions, movement, and game progress. However, they generally do not expose agent-side reasoning signals. For LuxLLM-Agent, the essential information includes LLM call status, cached intent, fallback reason, memory summary, opponent-risk metadata, and target rewrite events. These signals must be logged by the agent itself.

3.2 Season 1 Comparison

Season 1 provides a useful baseline for grid-based visualization and replay inspection. Its viewer style is effective for showing map state, unit positions, resource locations, city expansion, and movement over time. From this comparison, the project confirmed that visual inspection is important for debugging spatial game behavior.

However, the Season 1-style viewer is mainly replay-oriented. It explains what happened on the board, but it does not naturally explain why an agent selected a particular action. For the current project, this is a major limitation because the research target is not only replay visualization, but also LLM-assisted decision transparency. The Season 1 comparison therefore motivated the need to include agent-side metadata such as decision source, unit intent, fallback reason, memory state, and target-selection rationale.

Figure 1 shows the Season 1 comparison screenshot used during the cross-season viewer analysis. The image illustrates the strength of earlier replay-style visualization: it can show the board state, spatial layout, units, resources, and movement context. At the same time, it also highlights what is missing for the current project. A Season 1-style viewer does not naturally expose LLM decision source, cached intent, fallback status, opponent-risk metadata, or risk-filter target rewriting.

The Season 1 comparison also made clear that a visually clear map is not enough for explaining an LLM-assisted agent. A developer may be able to see that a unit moved to a tile, but not whether

the action came from a language-model suggestion, a cached instruction, a fallback rule, or a risk-filtered rewrite. This distinction is central to explainable LLM-agent systems. Therefore, the Season 1 comparison was useful not because it could be directly reused, but because it clarified the gap between replay visualization and agent explanation.

3.3 Season 2 Comparison

Season 2 provides a stronger comparison for long-horizon planning and resource-management complexity. Its factory, robot, lichen, and power-management mechanics show how a Lux agent can require multi-stage planning beyond immediate movement. This comparison helped clarify that a useful agent interface should expose not only unit positions, but also higher-level planning context and resource-related constraints.

At the same time, the Season 2 comparison showed that direct reuse of previous-season logic or visualization is limited. Season 2 focuses on factories, production chains, power, and territorial growth, while Season 3 focuses on relic discovery, energy-based movement, hidden map exploration, and opponent pressure. Therefore, the final system could not rely on direct reuse of a Season 1 or Season 2 viewer. It needed a Season 3-specific log-driven design.

Figure 2 summarizes the Season 2 comparison focus. Compared with Season 1, Season 2 places more emphasis on factories, robots, production, resources, power, and territorial growth. This helped motivate a key design decision in the current work: the viewer must expose planning context and internal agent metadata, not only final movement on the map.

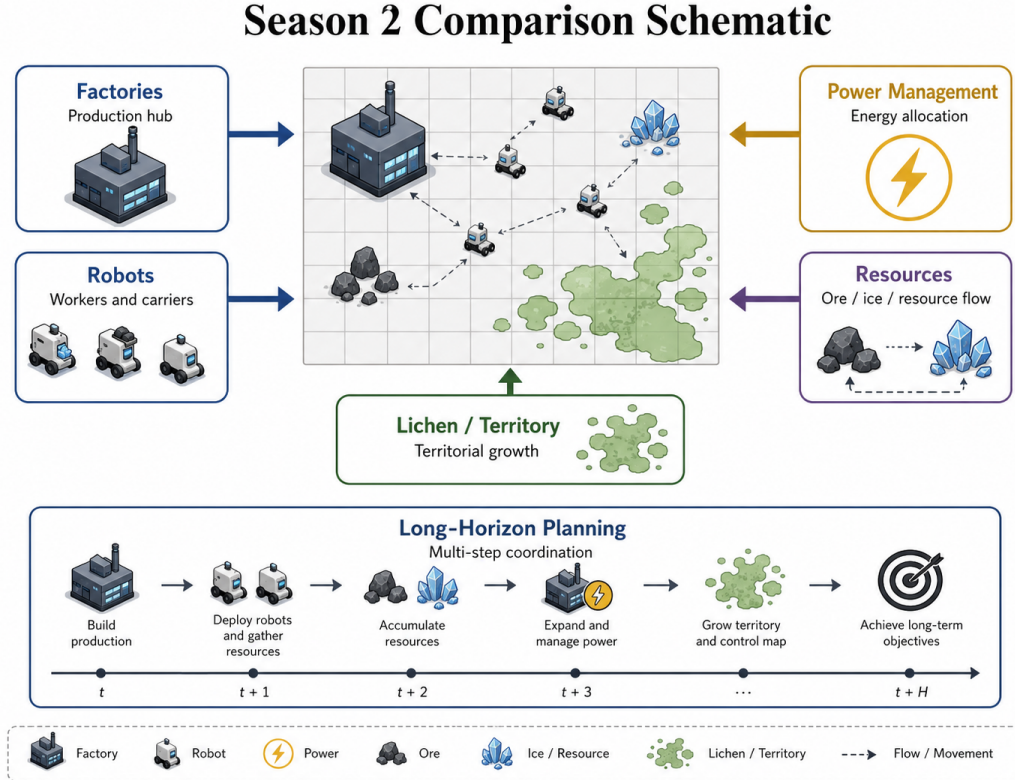


Figure 2: Season 2 comparison schematic used for cross-season viewer analysis. The figure summarizes the Season 2-style planning focus, including factories, robots, power management, resources, lichen or territory growth, and long-horizon coordination. It is used as a design comparison rather than as an output of the proposed Season 3 system.

The Season 2 comparison was especially important for the final viewer design. It suggested that explanation should not be restricted to official game objects. In Season 2, planning variables such as production intent or power constraints may not be obvious from a screenshot. Similarly, in Season

3, LLM intent, memory, risk filtering, and stale-tile selection are not visible unless the agent exports them. This directly motivated the use of `viewer_frames.json` as the central explanation artifact.

In summary, the Season 1 comparison showed the value and limitation of board-state replay visualization, while the Season 2 comparison emphasized the need to represent higher-level planning context. Together, these comparisons motivated the final Season 3 design: a log-driven GameView that does not only render the map, but also records and exposes the agent’s internal decision process.

3.4 Design Lessons from the Comparison

The cross-season comparison led to four design lessons. First, previous-season viewers are useful for visualizing board state, but they are insufficient for explaining LLM-assisted decisions. Second, each Lux season introduces different strategic abstractions, so a viewer designed for one season cannot be treated as a complete explanation tool for another. Third, the Season 3 agent requires a log-driven explanation pipeline because the relevant information includes not only official game state, but also internal agent metadata. Fourth, explanation should connect perception to action: if visible enemy pressure changes a unit target, that change should be recorded and displayed.

As a result, the final Season 3 GameView was designed around `viewer_frames.json`. Instead of only replaying official game states, the viewer reads frame-level records generated by the agent itself. These records include unit intents, LLM or fallback decision source, memory summaries, sensor and energy overlays, visible enemy units, opponent-risk analysis, and risk-filter events. This design directly follows from the Season 1 and Season 2 comparison tests: the project needed an explanation interface that goes beyond replay rendering and exposes the agent’s internal decision process.

These cross-season tests were used as engineering and design comparisons, not as formal performance benchmarks. The goal was to understand how much previous-season visualization and agent infrastructure could support the Season 3 explainability goal. Therefore, the comparison focuses on transferability, observability, and explanation coverage rather than win rate or score.

Table 2: Cross-season comparison and design implications for the final Season 3 system.

Comparison Target	What It Helped Test	Design Implication
Season 1	Grid-based replay visualization, map rendering, unit movement, and spatial debugging.	A visual interface is necessary, but replay rendering alone cannot explain LLM decisions, fallback behavior, or unit-level intent.
Season 2	Long-horizon planning, resource constraints, production logic, and higher-level strategic context.	A useful explanation interface should expose planning context and internal agent metadata, not only final movements.
Season 3	Relic discovery, hidden map exploration, energy-aware movement, multi-unit coordination, and opponent pressure.	The final system requires a Season 3-specific log-driven GameView that records memory, LLM trace, opponent risk, and risk-filter events.

4 System Overview

The system is organized as a sequential decision and logging pipeline. At each game step, the agent parses the Lux observation, updates persistent memory, summarizes the current state, obtains or reuses LLM-level intents, falls back to rule-based behavior when needed, converts intents into official Lux actions, applies risk-aware target filtering, and writes frame-level metadata for the viewer.

The final system should be understood as two connected layers. The first layer is the runtime agent, which must output legal actions under time and information constraints. The second layer is the explanation layer, which reconstructs the agent’s decision context from structured logs. This separation is important because the viewer does not control the game. Instead, it explains what the runtime did.

The main components are:

- **agent.py**: main agent loop, decision orchestration, opponent-unit logging, and frame-level logging.
- **action_planner.py**: conversion from unit intents to Lux actions, including opponent-aware and local-utility-aware risk filtering.
- **game_memory.py**: persistent memory for relic nodes, candidate tiles, explored tiles, energy tiles, and stale areas.
- **llm_decider.py**: local LLM interaction and structured intent generation.
- **rule_policy.py**: deterministic fallback for stable action generation.
- **s3_log_driven_gameview.html**: log-driven visualization and explanation interface.

The runtime exports structured frame data. This frame data includes game step, unit state, target information, visible tiles, memory summaries, LLM metadata, fallback status, visible enemy units, opponent-risk summaries, and risk-filter events. The viewer then renders these records in a form that is suitable for debugging, demonstration, and paper evidence.

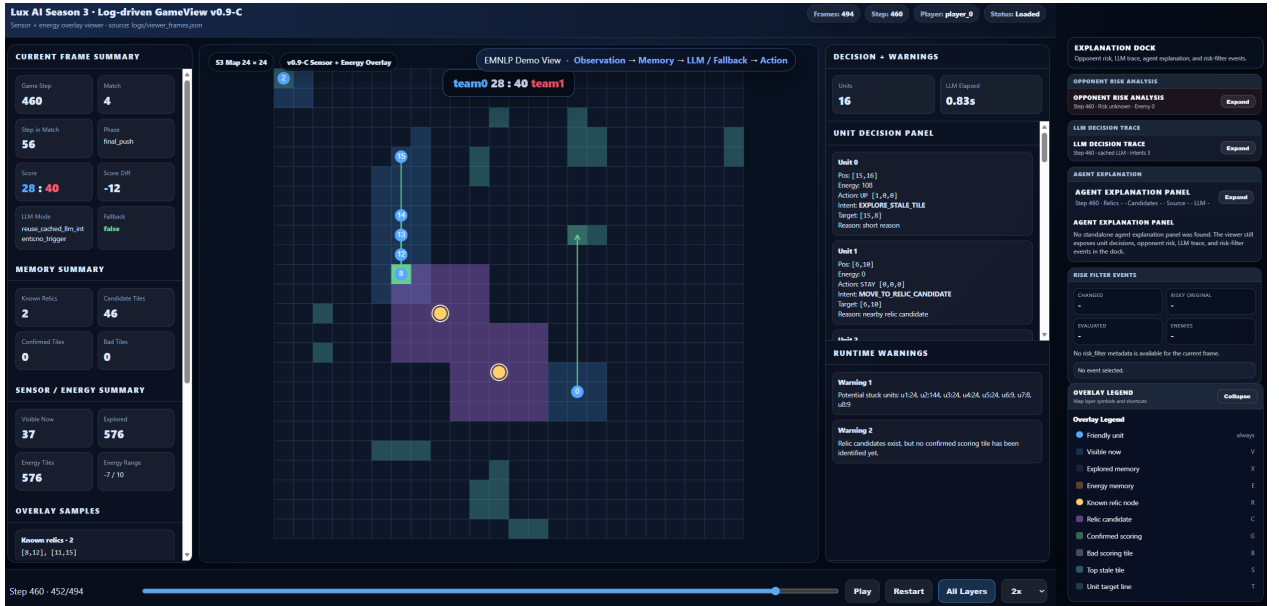


Figure 3: Final log-driven GameView interface. The viewer combines frame-level state summaries, memory and sensor overlays, unit target lines, decision panels, runtime warnings, and a docked explanation area. This layout is designed to make the agent’s observation, memory, decision source, and action execution inspectable without covering the main map.

5 Implementation Trajectory

The final package was built through a sequence of incremental engineering milestones. This trajectory is important because the final system combines several separate capabilities: a runnable Season 3 agent, LLM-assisted intent generation, fallback execution, structured logging, viewer reconstruction, opponent-risk analysis, and local-utility-aware filtering.

5.1 Runnable Season 3 Baseline

The first milestone was a runnable Lux AI Season 3 baseline that could participate in local matches and generate replay files. This provided the foundation for validating that the agent could execute legal actions in the official environment. At this stage, the most important requirement was stability rather than intelligence. The agent needed to load correctly, run through a match, and produce inspectable outputs.

5.2 LLM vs Rule Match Runner

The second milestone introduced an LLM-assisted player against a rule-based fallback opponent. Because local LLM calls can be slow or unreliable, the runtime was designed to support deterministic fallback and cached-intent reuse. This made the system more stable and allowed the agent to continue acting even when a fresh LLM decision was unavailable.

This stage established a practical division of responsibility. The LLM provides high-level intent, while deterministic code handles legality, target grounding, fallback behavior, and action conversion. This division is central to the system because it avoids relying on the LLM as a direct low-level controller.

5.3 Frame-Level Logging Pipeline

The third milestone introduced frame-level logging. Instead of relying only on console output or replay files, the agent began to export structured frame data for visualization. This made it possible to build a log-driven GameView that could expose memory, sensor coverage, energy information, unit intents, and target lines.

The frame log is a key design artifact. It allows the agent’s internal state to be inspected after the match, and it makes the viewer independent of ad-hoc console output. It also supports paper evidence, because specific frames can be selected and explained with corresponding metadata.

5.4 Explainability Panels

The fourth milestone added explainability panels to the viewer. These panels expose the decision source, LLM elapsed time, fallback status, unit-level intent traces, runtime warnings, and agent explanation text. This transformed the viewer from a map renderer into a debugging and explanation interface.

The layout also evolved during this stage. Floating panels were replaced with a more organized explanation dock. This improved the usability of the viewer for screenshots and demo recording because explanation cards no longer covered the main map.

5.5 Opponent-Aware and Local-Utility-Aware Filtering

The fifth milestone added opponent-aware risk analysis. Visible enemy units are recorded in the frame log and summarized through risk level, nearest-enemy distance, and contested candidate information. This makes opponent pressure inspectable and connects enemy visibility to action safety.

The final milestone added local-utility-aware risk filtering. The earlier risk filter could avoid unsafe targets, but diagnostics showed that some safe rewrites were overly distant from the original target. The updated filter preserves enemy safety while penalizing large target shifts and unnecessary extra unit travel. This reduced the far rewrite rate from 0.4160 to 0.0476 and the very far rewrite rate from 0.2880 to 0.0190.

6 LLM-Assisted Agent Design

The LLM is used as a high-level decision assistant. Instead of asking the model to generate exact low-level action arrays, the system asks for compact unit-level intents. This separation keeps the

LLM output interpretable and allows deterministic Python code to handle action legality, target selection, fallback behavior, and final action conversion.

Two engineering choices make the runtime stable. First, cached-intent reuse avoids querying the model at every frame and allows recent decisions to persist briefly. Second, deterministic fallback ensures that active units receive legal actions even when the LLM is unavailable, delayed, or produces unusable output. The viewer explicitly reports the decision source, distinguishing fresh LLM decisions, cached LLM state, fallback behavior, and rule-driven actions.

The intent representation is intentionally compact. Each unit can be assigned a high-level behavior such as exploration, movement toward a relic candidate, energy-related positioning, or fallback movement. The action planner then translates this intent into legal movement. This reduces the risk of malformed LLM output because the model does not need to generate exact game command arrays.

The design also makes debugging easier. If a unit moves unexpectedly, the developer can inspect whether the behavior came from a fresh LLM call, cached LLM intent, or fallback logic. This makes the LLM-assisted system more transparent than a purely black-box controller.



Figure 4: LLM decision trace example at Step 90. The viewer exposes whether the current decision comes from a fresh LLM call, cached LLM intent, or deterministic fallback. It also reports the model, elapsed time, unit intent count, and representative unit-level decisions, making the LLM-assisted control process easier to inspect.

7 Explainable GameView

The GameView reconstructs the agent’s behavior from `viewer_frames.json`. It visualizes the map, current visibility, explored memory, energy memory, relic nodes, relic candidates, unit locations, target lines, decision source, unit intents, runtime warnings, opponent risk, and risk-filter events.

The viewer is important because it turns raw match logs into an inspectable explanation chain. A user can see not only where a unit moved, but also what intent was assigned, whether the decision came from the LLM or fallback, what the memory state contained, whether visible enemies were nearby, and whether the target was rewritten by the risk filter.

The viewer contains four major regions. The left panel summarizes the current frame, including step, match phase, score, LLM mode, fallback state, memory summary, sensor coverage, and energy information. The central map renders the game board, units, visible tiles, memory overlays, and target lines. The right panel maps unit-level intents to actions and warnings. The far-right explanation dock contains opponent risk, LLM decision trace, agent explanation, risk-filter events, and

the overlay legend.

This organization supports both debugging and demonstration. During development, it helps identify why a unit acted incorrectly. During presentation, it provides a visual narrative: observation becomes memory, memory and LLM/fallback behavior become intent, intent becomes target selection, target selection may be filtered by opponent risk, and the final action is executed.

8 Opponent-Aware Risk Modeling

The opponent-aware component extracts visible enemy-unit positions from the observation and preserves them in frame logs. The viewer uses this information to display enemy pressure and nearest-enemy context, while the action planner uses it to judge whether a selected target is too close to visible enemy units.

This is not full opponent modeling. The system does not infer the opponent’s long-term strategy, learn an opponent model, or predict future enemy trajectories. Instead, it provides a transparent local-risk signal that can be used reliably inside the current runtime.

This distinction is important. Full opponent modeling would require estimating hidden state, predicting future movement, or learning opponent policy patterns. The current system is simpler and more explainable: it uses visible enemy units as local pressure signals. This is enough to support useful risk-aware filtering while keeping the mechanism understandable.



Figure 5: Opponent-aware risk analysis example. The viewer records visible enemy units and summarizes opponent pressure through risk level, nearest-enemy distance, and contested candidate information. This demonstrates that the system does not only track friendly units, but also incorporates visible opponent information into its explanation layer.

9 Risk-Aware and Local-Utility-Aware Action Filtering

The risk-aware action filter is the main functional extension beyond passive explanation. After an LLM, cached, or fallback policy selects a target, the planner checks whether the target is inside a visible-enemy risk radius. If the original target is risky and a safer alternative exists, the planner rewrites the target before producing the final Lux action.

The first risk filter successfully avoided risky targets, but diagnostics showed that some rewrites were safe but overly distant from the original target. The local-utility-aware update addresses this limitation with a target selector that balances opponent safety, original-target proximity, and

unit travel distance. The new selector still prefers targets outside visible enemy pressure, but also penalizes large shifts from the original target and unnecessary extra unit travel.

The filter records explicit events. Each event includes the unit id, intent, unit position, original target, filtered target, whether the target changed, nearest-enemy distance before and after filtering, visible enemy count, risk radius, and reason. This makes target rewriting auditable. A developer can inspect not only that the action changed, but also why it changed.

At Step 64, the frame log records two changed risk-filter events. Unit 5 has its target rewritten from [4,4] to [7,12], and Unit 6 has its target rewritten from [5,9] to [7,12]. In both cases, nearest-enemy distance improves from 1 to 5. This demonstrates that opponent-risk information is not only displayed in the viewer; it can directly affect final target selection.



Figure 6: Risk-aware target filtering evidence frame at Step 64. The opponent-risk panel reports high pressure, while the frame log records two changed risk-filter events. Unit 5 changes from target [4,4] to [7,12], and Unit 6 changes from [5,9] to [7,12]. In both cases, nearest-enemy distance improves from 1 to 5.

10 Evaluation and Evidence

This section summarizes the current experimental evidence for LuxLLM-Agent. The evaluation contains two complementary parts. First, we report a controlled benchmark of 57 completed Lux AI Season 3 matches across rule-based and LLM-assisted configurations. Second, we analyze frame-level diagnostic evidence from the log-driven viewer, focusing on opponent-risk signals and local-utility-aware risk filtering.

10.1 Controlled Benchmark Setup

To evaluate the effectiveness and runtime behavior of the proposed Lux AI Season 3 LLM-assisted agent, we conducted a controlled benchmark consisting of 57 completed matches across three configurations. Each configuration was evaluated over 19 matches.

Table 3: Experimental configurations used in the controlled benchmark.

Configuration	Matches
Rule vs Rule	19
Qwen2.5 1.5B vs Rule	19
Qwen2.5 7B vs Rule	19
Total	57

The rule-vs-rule setting is used as a control configuration. The two LLM-assisted settings compare different local language-model sizes under the same game environment. The objective is not to claim state-of-the-art Lux AI performance, but to examine whether LLM-assisted high-level intent generation improves match behavior and how model size affects runtime stability.

10.2 Win Rate Comparison

Table 4 reports the win rate of each configuration.

Table 4: Win rate comparison across controlled evaluation settings.

Agent Configuration	Matches	Wins	Losses	Win Rate
Rule vs Rule	19	9	10	47.37%
Qwen2.5 1.5B vs Rule	19	14	5	73.68%
Qwen2.5 7B vs Rule	19	13	6	68.42%

The rule-vs-rule benchmark produces a near-even win distribution, suggesting that the baseline setting is reasonably balanced. Both LLM-assisted configurations achieve higher win rates than the rule-only control. Qwen2.5 1.5B achieves a win rate of 73.68%, while Qwen2.5 7B achieves a win rate of 68.42%. These results provide initial evidence that LLM-assisted intent generation can improve match-level outcomes in this setting.

10.3 Score Comparison

Win rate alone does not fully describe agent behavior. Therefore, we also compare average match scores between player 0 and player 1.

Table 5: Average score comparison between player 0 and player 1.

Agent Configuration	Avg Score (P0)	Avg Score (P1)	Score Diff.
Rule vs Rule	2.421	2.579	-0.158
Qwen2.5 1.5B vs Rule	3.158	1.842	+1.316
Qwen2.5 7B vs Rule	3.158	1.842	+1.316

In the rule-vs-rule setting, the two players obtain similar average scores, which is consistent with the near-even win rate. When LLM-assisted decision making is introduced, the average score of player 0 increases and the score difference becomes positive. This suggests that the LLM-assisted agent can accumulate more game value than the rule-based baseline under the tested configurations.

10.4 Runtime and Computational Cost

Table 6 reports the runtime and LLM-related cost of each configuration.

Table 6: Runtime and computational cost comparison across agent configurations.

Agent Configuration	Avg Runtime (s)	Avg LLM Calls	Avg Fallbacks
Rule vs Rule	10.22	0.00	0.00
Qwen2.5 1.5B vs Rule	28.99	24.21	0.11
Qwen2.5 7B vs Rule	79.52	24.79	17.37

The results show a clear runtime trade-off. Qwen2.5 7B requires substantially more execution time than Qwen2.5 1.5B while using a similar number of average LLM calls. The 7B model also triggers more fallback events, with an average of 17.37 fallbacks compared with 0.11 for the 1.5B model. This indicates that larger models may provide richer reasoning capacity, but they can be less suitable for real-time game-agent control when latency and fallback stability are considered.

Table 7: Performance-efficiency trade-off between LLM model sizes.

Model	Win Rate	Avg Runtime (s)
Qwen2.5 1.5B	73.68%	28.99
Qwen2.5 7B	68.42%	79.52

Among the tested LLM-assisted configurations, Qwen2.5 1.5B provides the better balance between match performance, runtime cost, and fallback stability. This observation is important for LLM-agent system design: a larger model is not always preferable when the environment requires repeated decisions under time constraints.

10.5 Frame-Level Diagnostic Evaluation

In addition to the 57-match controlled benchmark, we analyze a representative log-driven run used by the current viewer and demonstration materials. This diagnostic evaluation focuses on whether the system records the decision chain and whether risk-aware filtering affects target selection.

The representative run is generated by the **v0.9-D2-controlled-runner** pipeline. The viewer data is stored in `logs/viewer_frames.json` and is derived from `logs/frame_log.jsonl`. The viewer selects `player_0` and contains 494 logged frames. The raw frame log contains 998 lines because it records both players.

The run contains 1196 risk-filter events and 210 changed target events. It also contains 81 frames with visible-enemy or opponent-risk signals, corresponding to 16.40% of the logged viewer frames. The system records enemy-distance metadata for the risk-filter events, allowing target changes to be inspected after the match.

Table 8: Frame-level diagnostic evidence from the representative log-driven run.

Metric	Value
Logged viewer frames	494
Raw frame-log lines	998
Visible-enemy signal frames	81
Visible-enemy signal rate	16.40%
Risk-filter events	1196
Changed target events	210

This evidence supports the system-level claim that LuxLLM-Agent does not only render game replay states. It records agent-side decision metadata, including opponent-risk information and target-filtering behavior, which can be inspected through the viewer and summary reports.

10.6 Local-Utility-Aware Risk Filter Analysis

The risk-aware action filter checks whether an intended target is too close to visible enemy units. If a target is risky, the planner can rewrite it to a safer alternative. However, an early version of this filter sometimes selected safer targets that were too far away from the original target. To address this, the local-utility-aware update adds a preference for safer targets that also preserve local movement utility.

Table 9 summarizes the effect of this update.

Table 9: Effect of the local-utility-aware risk filter on target rewriting.

Metric	Before	After	Reduction
Far rewrite rate	0.4160	0.0476	88.55%
Very far rewrite rate	0.2880	0.0190	93.39%
Average original-to-filtered target shift	9.41	4.41	53.14%

The local-utility-aware update substantially reduces overly distant target rewrites. The far rewrite rate decreases from 0.4160 to 0.0476, the very far rewrite rate decreases from 0.2880 to 0.0190, and the average original-to-filtered target shift decreases from 9.41 to 4.41. These results indicate that the updated filter preserves the safety benefit of opponent-aware target rewriting while reducing unnecessary target displacement.

10.7 Overall Findings

The current evaluation leads to four main findings.

First, LLM-assisted intent generation improves match outcomes in the controlled benchmark. Both LLM-assisted configurations outperform the rule-only control in win rate and average score.

Second, smaller language models can be more practical in real-time agent settings. Qwen2.5 1.5B achieves the highest win rate in this benchmark while requiring much less runtime than Qwen2.5 7B.

Third, the log-driven viewer provides inspectable frame-level evidence. The representative run contains hundreds of logged frames and more than one thousand risk-filter events, showing that the system records decision metadata beyond visual replay.

Fourth, local-utility-aware risk filtering improves target rewrite quality. It reduces far rewrites and average target shift while preserving opponent-risk-aware behavior.

10.8 Paper Relevance

These results directly support the future EMNLP demo paper framing. The controlled benchmark provides quantitative evidence for agent performance and runtime trade-offs. The diagnostic frame-level evaluation supports the system demonstration claim that the viewer exposes internal agent behavior. The local-utility-aware risk-filter analysis provides a concrete example of how the system can connect opponent-risk perception to action-level intervention and measurable improvement.

At the same time, the current evaluation should be interpreted as an initial system demonstration rather than a final large-scale benchmark. Future work will expand the evaluation to additional seeds, more agent variants, and larger multi-run comparisons.

11 Engineering Artifacts

The final project package contains both runtime artifacts and paper-oriented evidence artifacts. The runtime side includes the Lux AI Season 3 agent, the action planner, the memory module, the LLM decider, the deterministic fallback policy, the match runner, and the log-driven GameView.

The evidence side includes frame logs, risk-filter diagnostics, local-utility filter summaries, final consistency checks, screenshots, and a compiled demonstration paper.

This separation is important because the project is intended not only to run an agent, but also to make the agent’s behavior inspectable. The generated evidence files provide reproducibility support for the claims made in this paper. For example, the local-utility-aware filter improvement is supported by both JSON summaries and a human-readable Markdown report, while the viewer screenshots provide qualitative evidence of the explanation interface.

The final root directory was also cleaned so that core runtime files remain visible, while temporary patch scripts and development tools are archived. This makes the project easier to inspect and submit. The root retains the main agent files, the viewer, and the primary run scripts, while historical patch scripts are preserved in the archive for traceability.

12 Discussion

The strongest aspect of the current system is inspectability. The agent can be debugged and explained through frame-level evidence. This is especially important for LLM-assisted agents, where model output can otherwise appear opaque or disconnected from final actions. By separating high-level LLM intent from deterministic action planning, the system gains both interpretability and runtime stability.

The cross-season comparison also clarifies the value of the final design. Season 1-style visualization demonstrates the importance of spatial replay inspection. Season 2-style planning complexity demonstrates the need to expose higher-level context. Season 3 combines hidden information, energy constraints, relic discovery, and opponent pressure, making a log-driven explanation pipeline especially useful.

The controlled benchmark further shows that LLM-assisted intent generation can improve match-level performance under the tested configurations. However, the runtime results also show that larger models are not automatically better. Qwen2.5 7B is slower and triggers more fallback events than Qwen2.5 1.5B, suggesting that practical LLM-agent systems must balance reasoning capacity with inference cost and runtime reliability.

The risk-aware filter demonstrates a practical pattern for hybrid agents. Rather than asking the LLM to handle all tactical safety decisions, a deterministic module checks local opponent pressure and modifies unsafe targets. The local-utility-aware update improves this design by balancing opponent safety with local target utility.

This architecture suggests a broader design principle: LLMs can be useful inside game agents when they are used for interpretable intent generation, but the final system still needs deterministic grounding, validation, fallback, and explanation. The viewer is not a cosmetic add-on. It is the mechanism that makes this hybrid design inspectable.

12.1 Scalability Simulation

To evaluate scalability without overstating full-game throughput, we implemented a synthetic lightweight-worker simulation. The runner instantiates 10, 100, 500, and 1000 worker agents, each executing cached policy templates for 100 decision steps. The simulation does not launch full Lux AI Season 3 matches and does not invoke an LLM for every worker. Instead, five synthetic LLM calls represent sparse strategy-template generation. The 1000-worker setting completes 100,000 lightweight decisions in 0.807 seconds, while the LLM-call count remains fixed at 5. This result supports the intended scaling mechanism: expensive strategic reasoning is amortised across many low-cost worker decisions.

At the largest tested setting, the runner reaches 1000 lightweight workers and 100000 synthetic decisions. The synthetic LLM-call count remains fixed at 5, corresponding to 20000.0 worker decisions per LLM call at the largest setting. The 1000-worker run completes in 0.807 seconds with an estimated lightweight worker-state footprint of 9.894 MB.

Agents	Decisions	LLM calls	Cached uses	Fallback uses	Runtime (s)	Dec./s	Mem. (MB)
10	1000	5	1000	0	0.008	132,199.94	0.346
100	10000	5	10000	4	0.079	126,607.92	1.214
500	50000	5	50000	48	0.409	122,103.52	5.072
1000	100000	5	100000	81	0.807	123,915.78	9.894

Table 10: Architecture-level scalability simulation using lightweight worker agents. The runner uses cached policy templates and does not invoke an LLM once per worker.

Scalability limitation. This experiment is an architecture-level scalability simulation rather than a full Lux benchmark. The reported runtime measures lightweight policy execution and synthetic worker-state updates, not environment stepping, game-engine rendering, or 1000 complete Lux matches. We therefore use the result to support the system design claim that the LLM can act as a sparse strategist while many workers execute cached policy templates, rather than to claim full-game multi-agent performance at this scale.

13 Limitations

The system has clear limitations. First, the opponent-aware module is a local risk filter, not a learned or predictive opponent model. It uses visible enemy units as pressure signals, but it does not infer hidden opponent plans or predict future trajectories.

Second, the prompt has not yet been deeply optimized with few-shot examples, phase-specific reasoning templates, or self-checking. The LLM layer is functional, but the project does not claim that the prompt is optimal.

Third, the fallback policy remains conservative and does not yet implement sophisticated score-aware tactical choices. This is acceptable for a stable demonstration system, but it limits competitive strength.

Fourth, the project does not currently include reinforcement learning, tree search, self-play training, or value-function estimation. The evaluation therefore should not be interpreted as a full competitive benchmark.

Fifth, the cross-season tests were design comparisons rather than formal transfer-learning experiments. They show why Season 1 and Season 2 viewers or agent concepts are useful references, but they do not prove a quantitative relationship between seasons.

Sixth, the current controlled benchmark contains 57 matches, which is useful for initial system demonstration but still limited for strong statistical claims. Future evaluation should include more seeds, additional maps or configurations, and larger multi-run comparisons.

Finally, although the local-utility-aware update reduces distant rewrites, future work should further balance safety, local utility, task priority, score-aware tactics, and opponent prediction.

14 Ethics Statement

This project operates in a game environment and does not involve human subjects, private data, or real-world deployment decisions. The main ethical concern is accurate representation of capability. Because LLM-based systems can appear more general or more intelligent than they are, this paper avoids claiming full strategic reasoning, reinforcement learning, state-of-the-art performance, or complete opponent modeling.

The paper also avoids presenting the cross-season comparison as a formal benchmark. The comparison is used to motivate design decisions and explain why a Season 3-specific log-driven viewer is necessary. This distinction is important for maintaining a truthful description of the project.

The controlled benchmark is also presented as an initial system evaluation rather than a definitive competitive ranking. The results support the usefulness of LLM-assisted intent generation and log-



Figure 7: S1-style isometric battle replay viewer for Lux AI Season 3. The viewer loads merged replay frames and shows both players, unit positions, score, match state, timeline, and per-match score summary.

driven analysis in the tested environment, but they should not be interpreted as universal evidence that one model size or strategy is always superior.

14.1 S3 Battle Replay Viewer

To make the agent behaviour directly inspectable, the system includes an S1-style isometric battle replay viewer for Lux AI Season 3. The viewer loads merged replay frames and renders both players, unit positions, score state, timeline evidence, and match-level summaries in a floating-island visual style. This complements the diagnostic log viewer by providing a battle-replay interface suitable for live demonstration and paper figures.

During the demonstration, users can scrub through the replay, observe both `player_0` and `player_1` units, inspect score changes, switch into a recording-friendly Presentation Mode, and open a Final Result Overlay. Presentation Mode reduces auxiliary panels so that the floating-island battle map becomes the main visual focus for a short screencast.

The verified replay contains 505 frames, with both players present in 96.83% of frames. The replay records 662 risk-filter changes. Across 5 matches, `player_0` wins 3 matches and `player_1` wins 2 matches, with a total score of 407:363. The final frame reports `player_1` winning the final match 84:72.

The demo package includes the stable S3 isometric replay viewer, merged replay JSON, four paper-ready screenshots, a demo video, and verification reports. The stable viewer is stored at `docs/viewers/s3_isometric_battle_viewer_v09n12c3.html`, and the verified demo video is stored at `docs/demo_videos/Lux_S3_Isometric_Battle_Replay_v09n12.mp4`. The artifact index is stored at `docs/demo_artifact_index_v09n12g.md`.

The scalability artifacts include the O1 scalability direction plan, the O2 lightweight-worker simulation summary, and the O3 paper-ready scalability table. The main reports are stored at `docs/scalability_direction_plan_v09o1.md`, `docs/scalability_simulation_summary_v09o2.md`, and `docs/scalability_evaluation_table_v09o3.md`. The generated LaTeX table is stored at `docs/latex/scalability_table_v09o3.tex`.

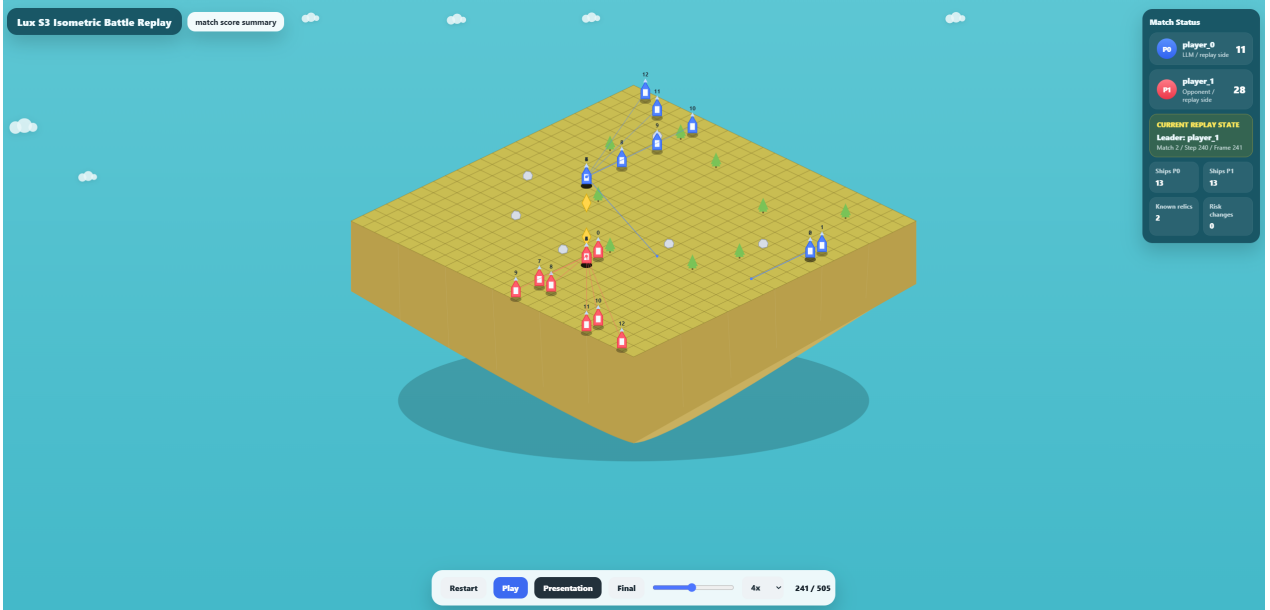


Figure 8: Recording-friendly Presentation Mode of the S3 battle replay viewer. Auxiliary panels are reduced so that the central floating-island battle map becomes the visual focus for demo-video recording.

15 Conclusion

LuxLLM-Agent shows how an LLM-assisted Lux AI Season 3 agent can be made stable, inspectable, and opponent-aware. The system combines LLM-level intents, deterministic fallback, persistent memory, log-driven visualization, opponent-risk extraction, risk-aware target filtering, and local-utility-aware target selection.

The project also demonstrates why Season 3 requires a different explanation pipeline from earlier Lux settings. Season 1 comparison highlights the value of spatial replay visualization. Season 2 comparison highlights the need to expose higher-level planning and resource context. Season 3 then motivates a log-driven GameView that records memory, LLM trace, opponent risk, and risk-filter events.

The evaluation provides both match-level and frame-level evidence. Across 57 controlled matches, the LLM-assisted configurations outperform the rule-only control in the tested setting, while runtime analysis shows that Qwen2.5 1.5B provides a better performance-efficiency balance than Qwen2.5 7B. The representative log-driven run further records 494 viewer frames, 1196 risk-filter events, 210 changed target events, and 81 visible-enemy signal frames. The local-utility-aware filter reduces far rewrite rate from 0.4160 to 0.0476 and average original-to-filtered target shift from 9.41 to 4.41.

Overall, the project contributes an explainable, opponent-aware, local-utility-aware LLM-agent demonstration system for Lux AI Season 3. It also provides a foundation for future EMNLP-style demo paper development, where the system can be further extended into a scalable log-driven framework for inspecting LLM-agent behavior across multiple runs and agent variants.

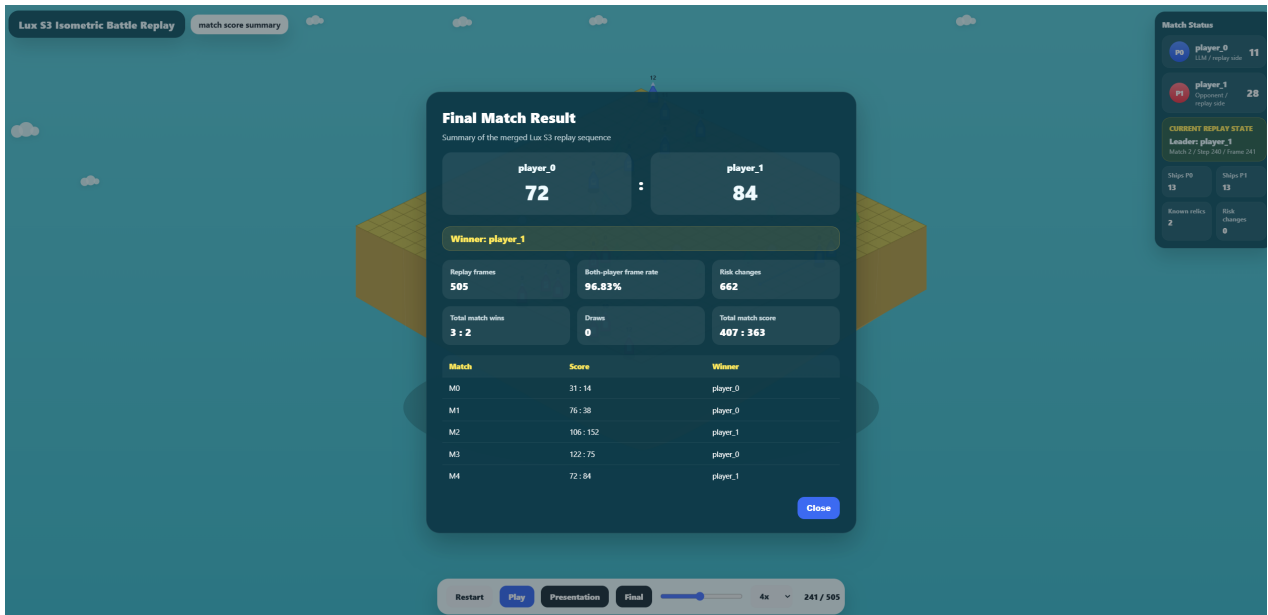


Figure 9: Final result overlay for the S3 replay. The overlay summarizes the winner, final score, replay coverage, risk-filter changes, total match wins, and total score.

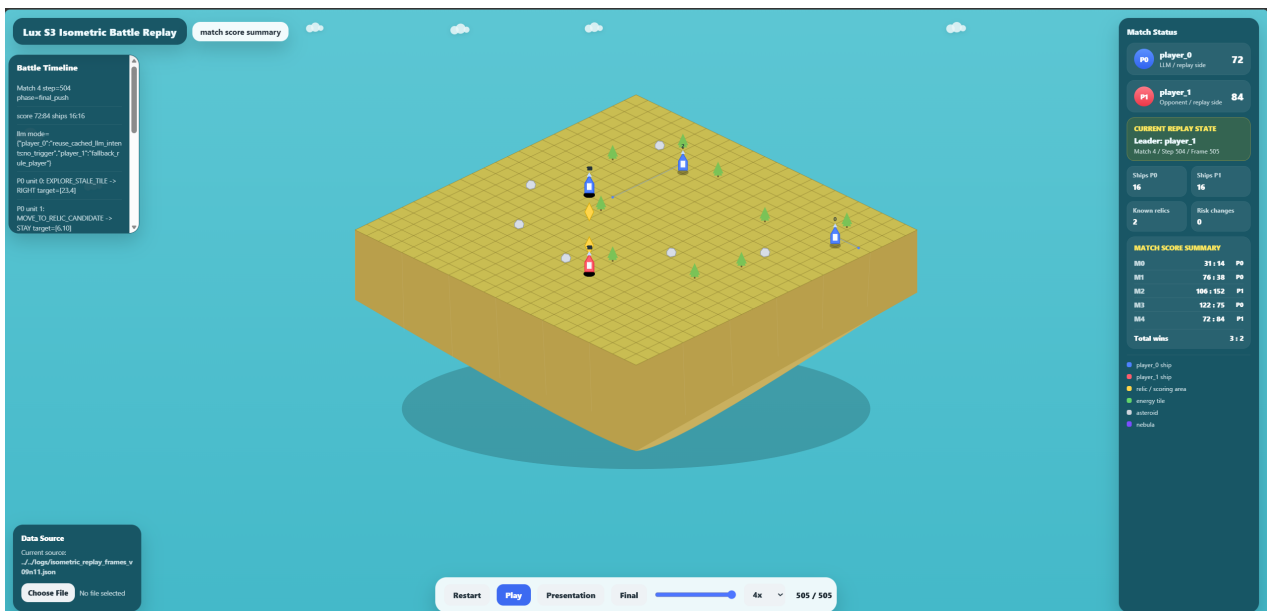


Figure 10: Per-match score summary in the S3 replay viewer. The panel displays individual match outcomes and total match wins over the replay sequence.